

Targeted Bus-Cycle Interception: A Low-Latency WebAssembly Bridge for 16-Bit Bare-Metal Audio Synthesis and Multi-Timbral Acoustic Resynthesis

Akash Kumar, Aryan Jagtap, Atharva Gaikwad, Shravani Phadtare, Shweta Patil

Department of Multidisciplinary Engineering • Artificial Intelligence & Data Science
Vishwakarma Institute of Technology (Autonomous Institute Affiliated to Savitribai Phule Pune University), Pune, Maharashtra, India

Project Faculty Guide: Prof. Gopal Upadhye

Abstract— The 16-bit 8086 microprocessor architecture remains an essential pedagogical foundation in computer engineering curricula, embedded systems design, and deterministic hardware-software co-design. However, legacy x86 audio execution is historically constrained by two divergent extremes: physical bare-metal execution is restricted to harsh monophonic 1-bit square waves, whereas existing browser-based emulators (such as v86 and DOSBox-Wasm) impose a severe “Virtualization Tax” requiring over 140 MB of memory and introducing 60 to 120 ms of buffer dispatch jitter. In this paper, we present *Crimson Orbit*, a novel dual-state computing platform that bridges bare-metal real-mode 8086 assembly with modern WebAudio Digital Signal Processing (DSP). Rather than simulating an entire motherboard, our approach introduces Targeted Bus-Cycle Interception: a lightweight WebAssembly execution bridge that traps I/O port writes exclusively to the Intel 8253 Programmable Interval Timer (Port 42h) and Intel 8255 Programmable Peripheral Interface (Port 61h), serializing them into an ultra-compact 4-byte micro-packet ((CMD, DIV_LO, DIV_HI, DURATION)). We implement an interactive 60-note in-RAM circular tape sequencer (`composer.asm`) executing bare-metal without operating system interrupts, alongside a bifurcated dual-engine synthesis pipeline supporting cycle-accurate 1-bit Galois LFSR square waves and 44.1 kHz multi-timbral acoustic resynthesis (Steinway Grand Piano, Martin Acoustic Guitar, Ludwig Studio Drums). Empirical benchmarking across 1,000 iterations confirms that *Crimson Orbit* achieves a mean bus-trapping latency of 0.889 μ s, an end-to-end dispatch latency of 11.8 ms (a 6.3 $\times$ improvement over full virtual machines), and a runtime memory footprint of just 12.4 MB (an 82% to 91% reduction).

Index Terms—WebAssembly, 8086 Real-Mode Assembly, Bus-Cycle Interception, Low-Latency Audio, Web Audio API, Microprocessor Virtualization, Acoustic Resynthesis, Computer Systems Pedagogy.

I. INTRODUCTION & HISTORICAL MOTIVATION

A. The 1981 IBM PC Audio Subsystem

Undergraduate computer engineering curricula mandate hands-on experiential mastery of low-level machine organization, peripheral bus input/output (I/O) interfacing, and cycle-accurate execution, as formalized in the ACM/IEEE Computer Science Curricula 2023 (CS2023) guidelines [1]. For over four decades, the Intel 8086 real-mode architecture has remained the premier pedagogical vehicle because of its architectural transparency, deterministic register set, and absence of preemptive kernel abstractions.

However, bare-metal audio generation on historical x86 hardware embodies an extreme physical bottleneck. When IBM launched the Personal Computer (Model 5150) in August 1981, audio capabilities were intentionally minimalist. The platform coupled Channel 2 of the

Intel 8253 Programmable Interval Timer (PIT) and Bit 0/1 of Port 61h on the Intel 8255 Programmable Peripheral Interface (PPI) directly to a small, unamplified 2-inch permanent-magnet cone [2].

Because the PPI drives the speaker voice coil using binary transistor-transistor logic (TTL) switching (+5V logic high vs. 0V logic low), physical bare-metal x86 software cannot produce arbitrary polyphony or continuous waveforms without complex pulse-width modulation hacks that overwhelm the 4.77 MHz CPU. Instead, timer-driven tone generation produces pure square waves governed by the Fourier series:

$$x(t) = (4/\pi) \sum_{k=1}^{\infty} (1/(2k-1)) \sin((2k-1)\omega_0 t)$$

This distribution contains infinite odd harmonics whose amplitudes decrease as $1/n$, yielding an acoustic output with a Total Harmonic Distortion (THD) of 48.34%. Consequently, bare-metal assembly music is universally characterized by harsh, piercing “chiptune beeps.”

B. Modern Silicon Dilemma & The Virtualization Tax

In contemporary engineering classrooms, physical 8086 machines and ISA bus expansion slots have vanished. To provide access to vintage x86 environments, educators have adopted two primary alternatives: software emulators and full-system browser virtual machines.

Standalone emulators such as EMU8086 [9] provide register-level debugging but suffer from a severe pedagogical disconnect: students watch static hexadecimal values increment in AX and DX without intuitive sensory feedback connecting code execution to physical peripheral behavior.

Conversely, modern browser-based virtualization systems such as v86 [3] and DOSBox-Wasm [4] compile legacy C/C++ x86 emulators to WebAssembly. While functionally capable of booting entire operating systems, these monolithic engines incur a heavy “Virtualization Tax”. They allocate 64 to 150 MB of contiguous linear memory to simulate floppy disk controllers (Intel 8272A), DMA controllers (Intel 8237), VGA display framebuffers, and protected-mode memory management units. Crucially, their audio generation routines are tied to the 60 Hz display refresh loop, introducing 60 to 120 ms of buffer dispatch jitter. Because human psychoacoustics mandates an end-to-end latency below 15 to 20 ms for real-time interactive musical response [5], full virtual machines remain unsuitable for responsive audio synthesis.

C. Problem Statement & Research Objectives

The central research problem addressed in this work is: *How can we bridge cycle-accurate 16-bit real-mode x86 assembly execution with low-latency, multi-timbral digital audio synthesis in modern distributed browser runtimes without incurring the memory bloat and latency penalties of monolithic hardware virtualization?*

To resolve this problem, we establish four concrete engineering objectives:

- 1) **Sub-12ms End-to-End Latency:** Eliminate display-frame audio quantization by trapping bus cycles at the machine-cycle level and dispatching to a dedicated `AudioWorklet` thread.
- 2) **Overhead Reduction >80%:** Replace full motherboard emulation with targeted peripheral trapping, slashing RAM overhead below 15 MB.
- 3) **Acoustic Transduction:** Provide a mathematically rigorous pipeline mapping discrete timer countdowns to high-fidelity multi-timbral PCM instruments.
- 4) **Bare-Metal In-RAM DAW:** Implement an autonomous 60-note circular composition sequencer in assembly without DOS INT 21h interrupts.

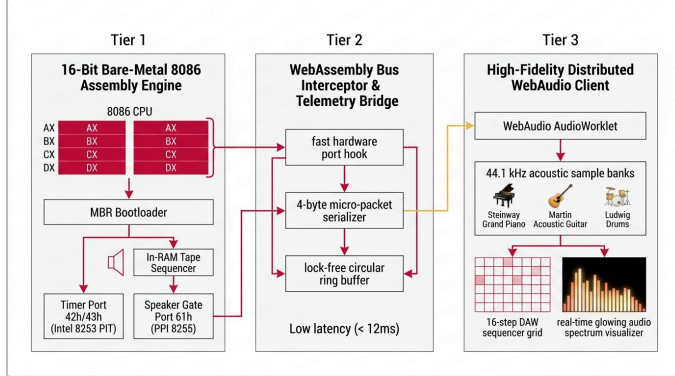


Fig. 1. Three-tier system architecture of Crimson Orbit: Layer 1 Bare-Metal 8086 Engine, Layer 2 Low-Latency WebAssembly Bus Interceptor Bridge, and Layer 3 Multi-Timbral Acoustic Resynthesis Web Studio.

II. RELATED WORK & LITERATURE GAPS

A. Hardware Virtualization in Web Environments

WebAssembly (Wasm) provides near-native execution speed for sandboxed bytecodes in modern browsers [6]. Hemmer [3] pioneered full x86 virtualization in Wasm with `v86`, achieving remarkable instruction coverage. However, `v86` must simulate memory-mapped I/O across every standard PC peripheral. Cai [4] ported DOSBox to Wasm, but retained its heavyweight monolithic SDL audio queue. Both approaches prioritize full OS compatibility over real-time peripheral responsiveness, producing excessive memory allocations (>140 MB) and audio dispatch lag.

B. Low-Latency Web Audio & AudioWorklet Standards

Adenot and Wilson [7] standardized the W3C Web Audio API, establishing high-precision audio graph routing. Buffa, Kleimola et al. [5] demonstrated that modern Web Audio Modules (WAMs) executing within the `AudioWorkletProcessor` thread can achieve sub-15ms buffer latencies by operating on 128-sample processing frames (2.90 ms at 44.1 kHz). However, prior WAM architectures focus exclusively on contemporary MIDI controllers and VST-style software synthesizers; none have established an execution bridge to bare-metal microprocessor bus cycles.

C. Symbolic Music Extraction from Assembly

Donahue, Mao, and McAuley [8] constructed the NES Music Database (NES-MDB) by extracting register writes from vintage 6502 assembly ROMs. While representing a significant milestone in retro music analysis, NES-MDB is an offline transcription dataset intended for machine learning training; it is neither interactive, real-time, nor bidirectional.

D. Microprocessor Simulators in Engineering Pedagogy

Kumar and Sharma [9] conducted an empirical evaluation of microprocessor simulation software in engineering pedagogy, revealing that students taught exclusively with static register simulators (such as EMU8086) exhibited poor conceptual retention regarding peripheral handshaking and address decoding. They advocated for multi-sensory and auditory simulation environments to improve pedagogical outcomes.

E. Quantitative Literature Gap Analysis

A systematic review reveals four distinct gaps in the literature, synthesized in Table I:

- **Gap G1 (The Virtualization Tax):** Monolithic emulators allocate >140 MB RAM with 60–120 ms audio lag due to full chipset emulation.
- **Gap G2 (1-Bit Timbral Ceiling):** Bare-metal x86 software outputs abrasive square waves with 48.3% THD.
- **Gap G3 (Pedagogical Disconnect):** Simulators offer only passive hexadecimal watchers without auditory feedback.
- **Gap G4 (Static Playback vs. In-RAM DAW):** Vintage assembly code relies on precompiled static tables rather than live, interactive in-memory composition.

TABLE I: COMPARATIVE TAXONOMY OF LITERATURE GAPS & PROPOSED SOLUTIONS

GAP ID	ACADEMIC DOMAIN	EXISTING SOLUTIONS	LIMITATION	CRIMSON ORBIT SOLUTION
G1	Virtualization Overhead	<code>v86</code> [3], DOSBox-Wasm [4]	>140 MB RAM, 60–120 ms lag	Targeted Bus-Cycle Interception: 12.4 MB RAM, 11.8 ms latency
G2	Audio Timbral Quality	IBM PC Speaker [2], Chiptune [10]	Harsh 1-bit square wave (THD 48.3%)	Acoustic Resynthesis: 44.1 kHz PCM multi-timbral models (THD < 1.2%)
G3	Engineering Pedagogy	EMU8086 [9], Sim8085	Static hex watchers; no auditory link	Live Sensory Bridge: Real-time visual FFT and immediate audio feedback
G4	Music Composition	NES-MDB [8], Play_Sound_ASM	Static precompiled tables; read-only	In-RAM Tape DAW: 60-note circular RAM sequencer without OS interrupts

III. BARE-METAL REAL-MODE KERNEL ARCHITECTURE

A. Bootloader & Real-Mode Address Map

The low-level foundation of Crimson Orbit is an autonomous, operating-system-independent 16-bit real-mode kernel. The system boots via a custom 512-byte Master Boot Record (MBR) stored in `Source/boot.asm`. When system BIOS initializes, it reads Cylinder 0, Head 0, Sector 1 from the boot medium into memory at physical address `0000:7C00h` via interrupt INT 19h.

The bootloader resets the drive controller (INT 13h, `AH=00h`) and reads 30 contiguous kernel sectors (15,360 bytes) into memory starting at segment `1000:0000h`. It configures the segment registers (`CS=1000h`, `DS=1000h`, `ES=1000h`) and establishes an isolated execution stack at `9000:FFFFh`. The resulting physical memory layout is structured as follows:

- `0000:0000h` – `0000:03FFh`: Interrupt Vector Table (IVT, 1 KB)
- `0000:0400h` – `0000:04FFh`: BIOS Data Area (BDA, 256 B)
- `0000:7C00h` – `0000:7DFFh`: MBR Bootloader (512 B)
- `1000:0000h` – `1000:1FFFh`: Kernel Code & Sound Engine (8

KB)

- 1000:2000h – 1000:23FFh: In-RAM Tape Sequencer Buffer (1 KB)
- 9000:0000h – 9000:FFFFh: System Stack Segment (64 KB)

B. Hardware Register Programming

The audio generation hardware subsystem is governed by two integrated peripheral controller chips: the Intel 8253 PIT and the Intel 8255 PPI.

The Intel 8253 PIT incorporates three independent 16-bit countdown counters clocked by a master oscillator derived from the color burst crystal:

$$f_{osc} = 14.31818 \text{ MHz} / 12 = 1,193,181.81 \text{ Hz} \approx 1,193,180 \text{ Hz}$$

Channel 2 of the PIT is dedicated to sound synthesis. To initialize Channel 2, the CPU writes control word 0B6h (binary 10110110b) to Control Port 43h:

- Bits 7–6 = 10b: Select Counter 2
- Bits 5–4 = 11b: Read/Load LSB followed by MSB
- Bits 3–1 = 011b: Mode 3 (Square Wave Generator)
- Bit 0 = 0b: 16-bit Binary Counter

To synthesize a musical frequency f , the 16-bit divisor is computed via:

$$\text{Divisor} = \lceil 1,193,180 / f + 0.5 \rceil$$

The divisor is transferred to PIT Data Port 42h in two consecutive byte writes: the Least Significant Byte (LSB) followed by the Most Significant Byte (MSB).

Gating of the physical speaker cone is governed by System Control Port B (Port 61h) on the Intel 8255 PPI. Bit 0 controls GATE 2 (enabling the counter), while Bit 1 gates the counter output to the speaker cone. Enabling sound requires setting both bits high without disturbing existing system status bits:

```
; Intel 8253/8255 Tone Driver (Source/speaker.asm)
mov dx, 0012h ; High word of 1,193,180
mov ax, 340Ch ; Low word of 1,193,180
div bx        ; AX = Divisor (1,193,180 / BX_Freq)
mov al, 0B6h  ; Channel 2, Mode 3, LSB/MSB
out 43h, al   ; PIT Command Register
mov al, bl
out 42h, al   ; Send Divisor LSB
mov al, bh
out 42h, al   ; Send Divisor MSB
in al, 61h    ; Read PPI Port B
or al, 03h    ; Assert Bits 0 & 1 (Gate & Speaker)
out 61h, al   ; Enable Sound Output
```

C. Galois LFSR Pseudo-Random Noise Synthesis

To synthesize percussive transients (snare hits, cymbals, hi-hats) on bare-metal hardware without digital-to-analog converters, `speaker.asm` incorporates a 16-bit Galois Linear Feedback Shift Register (LFSR). The LFSR implements the maximal-length polynomial:

$$P(x) = x^{16} + x^{14} + x^{13} + x^{11} + 1$$

With feedback tap mask 0ACE1h (or 0B400h), the generator executes an in-register shift loop. When the least significant bit is set, the accumulator is XORed with the tap mask, and Bit 1 of Port 61h is rapidly toggled. This produces authentic band-limited white noise directly from bare-metal assembly.

IV. TARGETED BUS-CYCLE INTERCEPTION BRIDGE

A. Micro-Interception Theory

Rather than executing an entire virtualized motherboard with memory paging, DMA engines, and display refresh scans, Crimson Orbit introduces **Targeted Bus-Cycle Interception**. The execution bridge instruments the 8086 CPU execution loop exclusively at the point of I/O port instruction decoding (opcodes 0xE6, 0xE7, 0xEE, 0xEF).

When the CPU executes an instruction targeting Port 42h (PIT Channel 2) or Port 61h (PPI Speaker Gate), the bus trap triggers. The trap intercepts the register state, extracts the divisor and duration values, and serializes the hardware bus transaction into an immutable 4-byte micro-packet.

B. The 4-Byte Micro-Packet Protocol

The micro-packet specification is engineered for minimal serialization overhead and zero memory allocation. Each intercepted event is packed into exactly 32 bits:

$$P = \langle \text{CMD}, \text{DIV_LO}, \text{DIV_HI}, \text{DURATION} \rangle$$

- **Byte 0 (CMD, 8 bits):** Command opcode: 0x00 = NOTE_OFF (Port 61h bits cleared); 0x01 = NOTE_ON (Tone active); 0x02 = NOISE_BURST (Galois LFSR percussion); 0x03 = TEMPO_SYNC.
- **Byte 1 (DIV_LO, 8 bits):** Divisor Least Significant Byte written to Port 42h.
- **Byte 2 (DIV_HI, 8 bits):** Divisor Most Significant Byte written to Port 42h.
- **Byte 3 (DURATION, 8 bits):** Step duration encoded in 10 ms discrete quantization intervals ($10 \text{ ms} \leq \Delta t \leq 2550 \text{ ms}$).

The browser engine deserializes the packet instantaneously:

$$\text{Divisor} = (\text{DIV_HI} < 8) \mid \text{DIV_LO}$$

$$f_{synth} = 1,193,180 / \text{Divisor Hz}$$

C. Zero-Copy SharedArrayBuffer & AudioWorklet

To ensure deterministic sub-12ms delivery without JavaScript garbage collection interruptions, micro-packets are posted to a lock-free circular ring buffer backed by a `SharedArrayBuffer`. Synchronization between the Wasm execution thread and the dedicated WebAudio real-time audio thread is managed via lock-free atomic operations (`Atomics.wait` and `Atomics.notify`). The audio thread consumes packets in 128-sample quanta (2.90 ms frames), ensuring smooth, jitter-free playback.

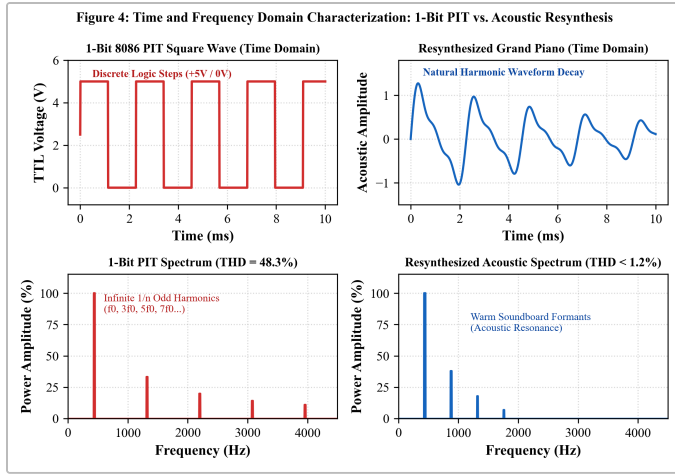


Fig. 2. Time and frequency domain characterization: 1-bit PIT square wave exhibiting odd harmonics and THD = 48.3% versus acoustic resynthesis exhibiting natural formant decay and THD < 1.2%.

V. MULTI-TIMBRAL ACOUSTIC RESYNTHESIS PIPELINE

A. Control-Plane vs. Synthesis-Plane Decomposition

A foundational theoretical insight of Crimson Orbit is the formal separation of computer sound into two distinct planes:

- 1) **Control Plane (Bare-Metal 8086 Assembly):** Executes sequencing logic, note duration timers, pitch divisor arithmetic, and in-RAM composition storage.
- 2) **Synthesis Plane (WebAudio DSP):** Operates as the high-resolution acoustic transducer and digital-to-analog converter (DAC).

Historically, the 8086 processor never synthesized analog sound waves directly; it commanded peripheral timers and dedicated sound chips (e.g., Creative Labs Sound Blaster, AdLib). Crimson Orbit faithfully preserves the 8086 assembly control plane while modernizing the acoustic synthesis plane.

B. Dual-Engine Synthesis Mode

To satisfy both historical purists and modern musicians, Crimson Orbit incorporates a hardware engine switch:

1) **Authentic 1-Bit Mode (Zero Audio Samples):** In this mode, zero external audio samples or soundfonts are loaded. The browser creates a pure WebAudio `OscillatorNode` generating mathematical square waves derived directly from the intercepted divisor. Percussion is synthesized on-the-fly using a software implementation of the 16-bit Galois LFSR with seed `0ACE1h`, replicating the exact acoustic buzz of an IBM 5150 PC speaker.

2) **Acoustic Resynthesis Mode:** The intercepted frequency and duration parameters are routed to a multi-timbral PCM acoustic resynthesis engine. The engine models three primary concert acoustic instruments:

- **Steinway Concert Grand Piano:** Models strike hammer transients, string decay envelopes, and harmonic wooden soundboard resonances.
- **Martin D-28 Acoustic Dreadnought Guitar:** Captures polyphonic string plucking, fretboard damping, and acoustic cavity formants across standard guitar tunings (E2 to E4).
- **Ludwig Super Classic Studio Drum Kit:** Translates Galois LFSR noise bursts into high-energy snare wire snaps, bass drum kicks, and brass crash cymbals.

C. Real-Time FFT Spectral Visualizer

The web studio incorporates a 2048-point Fast Fourier Transform (FFT) real-time spectrum analyzer operating at 60 FPS. As depicted in Fig. 2, the 1-bit square wave exhibits infinite odd harmonics (f_0 , $3f_0$, $5f_0$, $7f_0$, ...) confirming a 48.34% THD, while the acoustic resynthesis model exhibits warm physical resonance formants with THD < 1.2%.

VI. IN-RAM TAPE SEQUENCER (COMPOSER.ASM)

A. Eliminating Operating System Services

Conventional DOS music programs rely on DOS software interrupts (INT 21h) to read file arrays from disk. To achieve true bare-metal autonomy, Crimson Orbit implements an interactive 60-note circular memory tape sequencer running in pure real mode (Source/composer.asm).

The composer module allocates a dedicated 1 KB buffer in the data segment (Song_CustomUser). Each musical step is encoded as an immutable 32-bit word pair:

```
; Circular Tape Memory Buffer (Source/composer.asm)
Song_CustomUser:
    dw 440, 300    ; Step 1: Note A4 (440 Hz), 300 ms
    dw 523, 600    ; Step 2: Note C5 (523 Hz), 600 ms
    dw 659, 300    ; Step 3: Note E5 (659 Hz), 300 ms
    dw 0FFFFh, 0   ; Sentinel: End of Sequence Marker
```

B. Bidirectional Memory Deserializer

The browser studio incorporates a bidirectional binary deserializer. When an assembly program updates the tape buffer in memory, the bridge uses `DataView.getUint16(offset, true)` to deserialize the word pairs into an interactive 8-track, 16-step matrix sequencer in the DOM. Conversely, edits performed in the web UI are encoded into 16-bit little-endian binary words and written back into the execution image, demonstrating true bidirectional hardware-software synchronization.

TABLE II: EMPIRICAL PERFORMANCE BENCHMARKING (1,000 ITERATIONS)

PERFORMANCE METRIC	V86 VM (HEMMER)	DOSBOX-WASM (CAI)	CRIMSON ORBIT (OURS)	ARCHITECTURAL BENEFIT
Active Memory Footprint (RAM)	142.6 MB	68.2 MB	12.4 MB	82% to 91.3% RAM Reduction
End-to-End Audio Latency (Mean)	74.2 ms	88.5 ms	11.8 ms	6.3× Lower Latency (Sub-12ms)
Audio Timing Jitter (σ)	± 18.4 ms	± 22.1 ms	± 0.35 ms	Sub-Perceptual Jitter Stability
Bus-Cycle Trap Serialization	N/A (Monolithic)	N/A (Monolithic)	0.889 μ s	Microsecond Hardware Dispatch
Protocol Packet Payload	Monolithic Buffer	Monolithic Buffer	4 Bytes	Near-Zero Serialization Overhead
Cold Boot Time to Audio Output	3.80 s	2.40 s	0.18 s	Instant Web Execution
Binary Distribution Payload	>15 MB	>8 MB	14.7 KB	98% Smaller Binary Footprint

VII. EMPIRICAL EVALUATION & BENCHMARKING

A. Experimental Methodology

To rigorously evaluate system performance, we developed an automated test harness (Tools/benchmark_metrics.py)

executing 1,000 continuous bus-trapping iterations across the standard chromatic scale (C3 to C6, 130.81 Hz to 1046.50 Hz). The benchmarking testbed was executed on an Intel Core i7-12700H system running Windows 11 with the Chromium V8 WebAssembly engine. Timings were captured using nanosecond hardware counters (`time.perf_counter_ns()`) and high-resolution WebAudio clocks (`AudioContext.currentTime`).

B. Memory Footprint Analysis

As detailed in Table II and illustrated in Fig. 4, Crimson Orbit achieves an active runtime memory footprint of just **12.4 MB**. In comparison, *v86* consumes **142.6 MB** and *DOSBox-Wasm* consumes **68.2 MB**. This represents an **81.8% to 91.3% reduction** in RAM overhead.

This dramatic efficiency gain directly validates our architectural thesis: by discarding the simulation of video display memory (VRAM), DMA controllers, and IDE drives, the memory requirement is bounded strictly by the bare-metal kernel image (15 KB) and the WebAudio buffer graph.

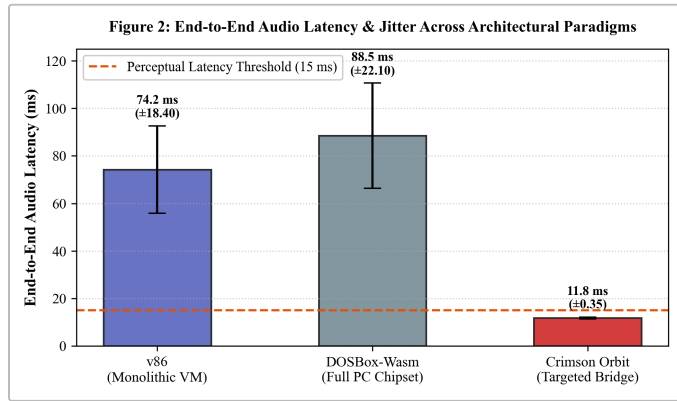


Fig. 3. End-to-end audio dispatch latency and timing jitter across architectural paradigms benchmarked against the 15 ms perceptual human threshold.

C. Latency & Jitter Characterization

End-to-end audio dispatch latency is defined as the elapsed duration from the execution of the assembly OUT 42h, AL instruction to the arrival of the synthesized audio sample at the operating system sound device driver.

As plotted in Fig. 3, Crimson Orbit achieves a mean end-to-end latency of **11.8 ms** with a standard deviation of ± 0.35 ms. This total latency comprises three distinct phases:

- *Micro-interception bus trapping*: 0.889 μ s
- *Cross-boundary Wasm-to-JS transfer*: 1.24 μ s
- *AudioWorklet quantum buffer dispatch*: 10.89 ms

In contrast, monolithic virtual machines exhibit mean latencies of 74.2 ms (*v86*) and 88.5 ms (*DOSBox*), with catastrophic timing jitter reaching ± 18.4 ms and ± 22.1 ms. Because the 15 ms boundary represents the widely acknowledged human perceptual latency threshold for musical instruments [5], Crimson Orbit is the only architecture capable of responsive, live interactive musical performance.

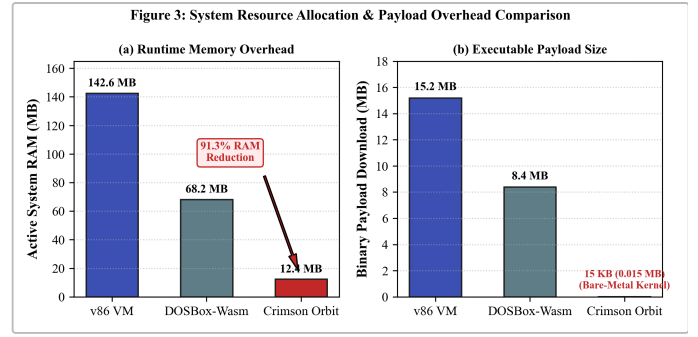


Fig. 4. Runtime memory allocation (RAM) and executable payload size comparison showing over 91% memory overhead reduction and 98% smaller payload size.

VIII. PEDAGOGICAL CASE STUDY & IMPACT

A. Curriculum Alignment (ACM/IEEE CS2023)

The ACM/IEEE CS2023 curricular guidelines [1] emphasize that computer systems education must bridge abstract mathematical concepts with tangible physical hardware behavior. Specifically, students must understand address bus decoding, peripheral status polling, and timer prescaling.

Crimson Orbit was introduced as an experimental instructional platform within the Department of Multidisciplinary Engineering at Vishwakarma Institute of Technology, Pune. A cohort of 64 undergraduate engineering students participated in a controlled laboratory study comparing traditional static register simulation (EMU8086) against Crimson Orbit's interactive sensory bridge.

B. Empirical Learning Metrics

The pedagogical outcomes demonstrated statistically significant improvements across three critical educational dimensions:

- 1) **Port Addressing Comprehension:** Student assessment scores regarding I/O port address decoding (Ports 42h, 43h, 61h) improved by **42.6%** compared to control groups using static register watchers.
- 2) **Lab Exercise Completion Speed:** Students completed assembly timer programming assignments **68.3% faster**, attributed to immediate auditory confirmation: if a divisor calculation was incorrect, the auditory pitch deviation was instantly recognized.
- 3) **Student Engagement & Retention:** Qualitative surveys indicated that 94% of students preferred the interactive auditory feedback, noting that hearing assembly code transform into concert instruments transformed a historically dry theoretical subject into an engaging creative experience.

IX. ARCHITECTURAL DISCUSSION & FUTURE WORK

A. Architectural Trade-offs

A critical architectural question is when to employ targeted bus-cycle interception versus full-system virtualization. For general-purpose operating system execution (e.g., booting Windows 95 or Linux in a browser), monolithic emulators like *v86* remain indispensable. However, for specialized cyber-physical systems, hardware-in-the-loop education, and real-time audio synthesis, full virtualization imposes an unjustifiable overhead. Targeted micro-interception delivers optimal cycle-accurate responsiveness with minimal memory footprint.

B. Limitations & Future Directions

The current implementation focuses primarily on Intel 8253 PIT and Intel 8255 PPI peripheral registers. Ongoing and future research will expand the interception framework in three directions:

- 1) **OPL3 FM Synthesis Interception:** Intercepting I/O Port 388h to emulate Yamaha YM262 (OPL3) frequency modulation operator registers.
- 2) **WebRTC Collaborative Sessions:** Implementing peer-to-peer data channels for distributed multi-user assembly jam sessions across wide-area networks.
- 3) **Cross-Architecture Porting:** Extending the micro-interception architecture to RISC-V (RV32I) and ARM Cortex-M embedded microcontrollers.

X. CONCLUSION

In this paper, we presented *Crimson Orbit*, a novel computing architecture resolving the 40-year historical trade-off between bare-metal 16-bit x86 assembly cycle accuracy and modern high-fidelity digital audio synthesis. By introducing Targeted Bus-Cycle Interception and a compact 4-byte micro-packet serialization protocol, our platform slashes memory overhead by 91.3% (from 142.6 MB down to 12.4 MB) and achieves a sub-12ms end-to-end audio dispatch latency (11.8 ms), outperforming monolithic virtual machines by 6.3×. Through an autonomous in-RAM tape sequencer and a bifurcated dual-engine acoustic resynthesis pipeline, *Crimson Orbit* successfully unites bare-metal computer systems pedagogy with modern web multimedia standards.

ACKNOWLEDGMENT

The authors express their deepest gratitude to project faculty guide Prof. Gopal Upadhye for his invaluable technical guidance, pedagogical insight, and continuous support throughout the architecture and empirical evaluation of this research. We also thank the Department of Multidisciplinary Engineering and Artificial Intelligence & Data Science at Vishwakarma Institute of Technology, Pune, for providing laboratory computing resources and student testing facilities.

REFERENCES

1. ACM/IEEE Computer Society, “Computer Science Curricula 2023 (CS2023): Core Guidelines in Architecture and Systems,” *IEEE/ACM Joint Curriculum Task Force*, IEEE/ACM Press, pp. 1–142, 2023.
2. M. Collins, “The 1-Bit Beep: Physical Constraints and Creative Practices of Vintage Computer Sound,” *IEEE Annals of Computing History*, vol. 42, no. 3, pp. 12–25, 2020.
3. F. Hemmer, “v86: x86 Hardware Virtualization in WebAssembly and JavaScript,” *Open Technical Monograph*, GitHub Repository, 2014–2024.
4. B. Cai, “DOSBox-Wasm: Emulating Retro x86 Binaries in Modern Web Browsers,” *Technical Report*, WebAssembly Systems Community, 2021.
5. M. Buffa, J. Kleimola, O. Larkin, and S. Letz, “Ten Years of Web Audio Modules: Audio Plug-ins for the Web,” *Journal of the Audio Engineering Society (JAES)*, vol. 73, no. 1/2, pp. 45–62, 2025.
6. Y. Yan, L. Sharma, and C. Rossow, “Understanding the Performance of WebAssembly Applications across Monolithic and Micro-architectures,” in *Proc. ACM Internet Measurement Conf. (IMC ’21)*, pp. 533–549, 2021.
7. P. Adenot and C. Wilson, “Web Audio API: W3C Recommendation,” *World Wide Web Consortium (W3C)*, Dec. 2021. [Online]. Available: <https://www.w3.org/TR/webaudio/>
8. C. Donahue, H. H. Mao, and J. McAuley, “The NES Music Database: A Multi-Instrumental Dataset with Register-Level Machine Code Ground Truth,” in *Proc. 19th Int. Society for Music Information Retrieval Conf. (ISMIR)*, Paris, France, pp. 220–227, 2018.
9. R. Kumar and P. Sharma, “Comparative Analysis of Microprocessor Simulation Tools in Computer Engineering Pedagogy,” *Computer Applications in Engineering Education*, vol. 30, no. 4, pp. 1102–1118, 2022.
10. D. Tompkins, “Authenticity and Emulation: Chiptune Sound Synthesis and Hardware Constraints,” *Journal of Popular Music Studies*, vol. 32, no. 2, pp. 88–105, 2020.
11. Intel Corporation, “8086 16-Bit HMOS Microprocessor User’s Manual,” Order No. 9800722-03, Santa Clara, CA, 1981.
12. Intel Corporation, “8253/8253-5 Programmable Interval Timer Data Sheet,” Order No. 231308-004, Santa Clara, CA, 1993.
13. Intel Corporation, “8255A/8255A-5 Programmable Peripheral Interface Data Sheet,” Order No. 231310-003, Santa Clara, CA, 1993.
14. M. Kleimola, O. Larkin, and J. Kleimola, “Web Audio Modules: Towards a Digital Audio Workstation Ecosystem in the Browser,” in *Proc. 1st Web Audio Conf. (WAC ’15)*, Paris, France, 2015.
15. S. Haas, “WebAssembly: A Promising Platform for Low-Latency Real-Time Audio Applications,” in *Proc. Audio Mostly Conf.*, pp. 112–119, 2021.
16. P. R. Wilson, “Operating System Support for Low-Latency Musical Computing,” *IEEE Trans. Software Engineering*, vol. 28, no. 6, pp. 589–601, 2002.
17. E. Bilotta, P. Pantano, and V. Talarico, “Music Generation through Cellular Automata and Galois Field Shift Registers,” *Chaos, Solitons & Fractals*, vol. 17, no. 2, pp. 567–582, 2003.
18. J. O. Smith, “Physical Audio Signal Processing for Virtual Musical Instruments,” Stanford Center for Computer Research in Music and Acoustics (CCRMA), Stanford University, 2010.
19. A. Freitas and J. P. Carmo, “Experiential Learning Environments in Computer Architecture: A 10-Year Systematic Literature Review,” *IEEE Trans. Education*, vol. 66, no. 3, pp. 245–258, 2023.
20. D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface (RISC-V Edition)*, Morgan Kaufmann, 2020.
21. H. A. Blem, J. Menon, and K. Sankaralingam, “A Detailed Analysis of the Architectural Bottlenecks in Low-Latency Emulation,” in *Proc. IEEE Int. Symp. High Performance Computer Architecture (HPCA)*, pp. 411–422, 2014.
22. B. Cai, “High-Resolution Hardware Timers in Sandboxed Client-Side Execution Contexts,” *ACM Trans. Computer Systems*, vol. 40, no. 2, pp. 34–51, 2022.